Digital Library Helpdesk Prototype

Prototype Documentation

Markus Tiedemann
Nathan Hagos Berhe
Abel Turlej

The Lab
Group 11
ACS
2025/26

March 2026

Table of Contents

What Is This Prototype 3
What it Does 3
What it does NOT do (by design) 3
Code Repositories 3
Architecture Overview 4
Components at a Glance 5
How Chatbot, Ticketing and Microsoft Are Connected 6
User flow (Authenticated) 6
What Goes Into a Ticket 7
Guest Users 7
Microsoft Teams 7
The AI Model 7
Which model? 7
Why this model? 8
Advantages 8
Limitations 8
Choosing an AI Model 9
Privacy & Data Sovereignty 9
Language Support 9
Cost Model 9
Accuracy vs Speed Trade-off 10
Integration Complexity 10
How the AI Was Configured 10
System Prompt | The AI's Instructions 10
Knowledge Base | The AI's Reference Library 10
When a user sends a query: 11
Intent Detection | What Is the Student Asking? 11
Each intent contains: 12
Intent Example 12
Example intent structure: 12
Examples include: 12
Escalation Rules | When to Involve a Librarian? 13
DMN determines: 13
Language Detection 13
Storage Architecture 13
Ingestion Process 14
System Flow (Combined) 14
Design Summary 15
Setup & Configuration 15
What Is Needed to Run This 15

Environment Configuration....15
Setup options....17
Configuration....17
Microsoft Lists....17
Required Lists columns....17
Three views to configure....18
Microsoft SharePoint....18
Setup....18
Document content....18
Microsoft Power Automate Flows....18
Flow 1....19
Flow 2....20
If True....21
If False....22
Flow 3....22
Starting the Services (Development)....23
Start the AI service (port 8001)....23
Start the backend (port 8000)....23
Start the frontend (port 5173)....23
Pre-configured Accounts....23
Registering....23
IMPORTANT – Development / Test Only:....23
Current Status....24
Known Limitations of the Prototype....24
Technology and Technique Glossary....25
Extras....26
Flow 1 JSON....26
Flow 1 Parameters 1.1....27
Flow 1 Parameters 1.2....27
Flow 1 Parameters 1.3....27
Flow 2 JSON....29
Flow 2 Parameters....29
Flow 3 JSON....31

Documentation

What Is This Prototype

The KdG Digital Library Helpdesk is a **web-based chatbot** that helps library users (students, staff and other authorized users) get answers to common library questions without needing to contact a librarian directly.

What it Does

1. A student visits the helpdesk website and types their question (e.g., "When does the library close?" or "Why is my account blocked?")
2. An AI assistant answers immediately if it can
3. If the question is too complex, sensitive, or the AI is not confident enough, it escalates **to a librarian**
4. A ticket is automatically created in **Microsoft Lists**, and the librarian gets notified
5. The librarian can take over the conversation live, in real time

What it does NOT do (by design)

1. It does not access the library management system
2. It cannot modify accounts, confirm real-time book availability, or make reservations
3. It does not replace librarians
    a. It filters and routes questions, so librarians spend time on what matters

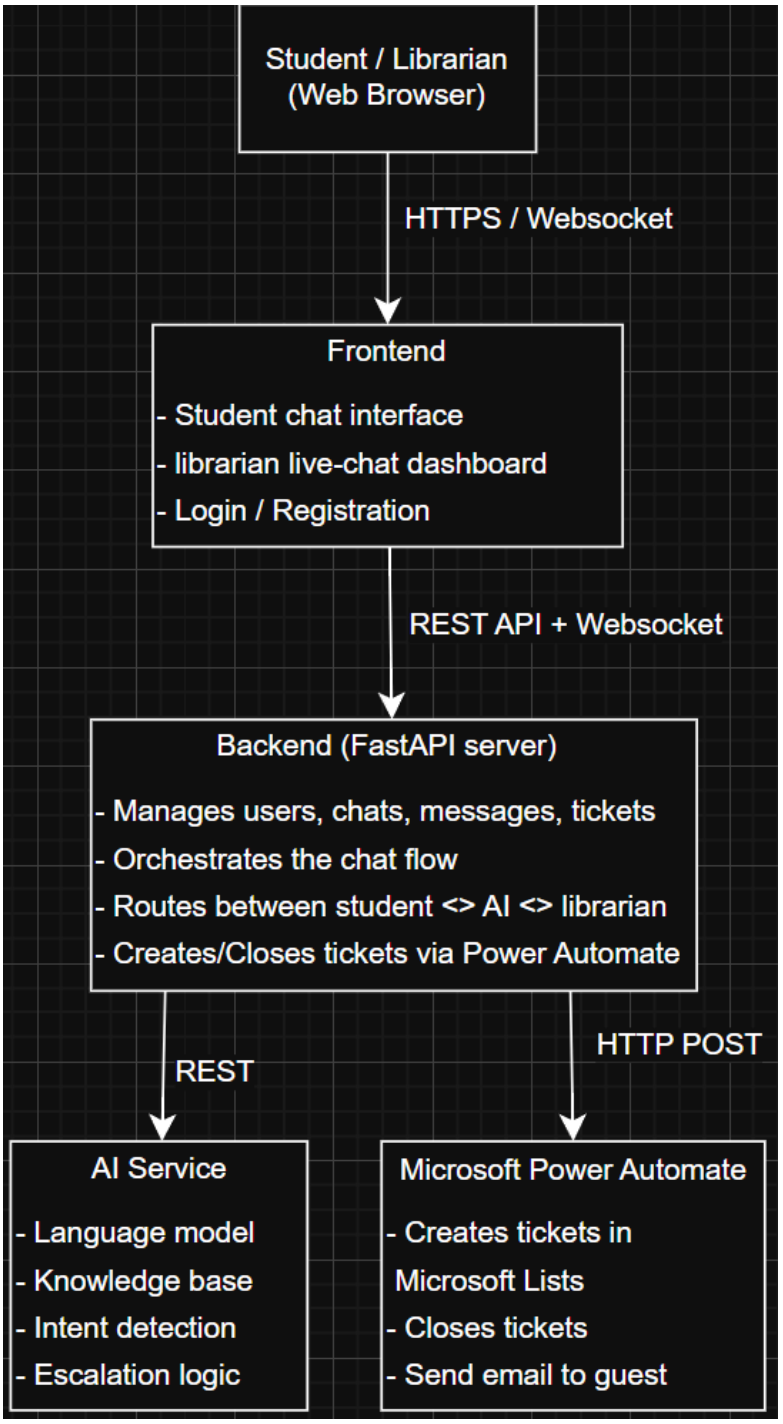
Campuses covered (MVP): Campus Zuid and Campus Groenplaats

Code Repositories

BACKEND: https://gitlab.com/kdg-ti/the-lab/teams-25-26/11_man/backend
FRONTEND: https://gitlab.com/kdg-ti/the-lab/teams-25-26/11_man/frontend
AI: https://gitlab.com/kdg-ti/the-lab/teams-25-26/11_man/ai

Architecture Overview

The system consists of three separate services that work together:



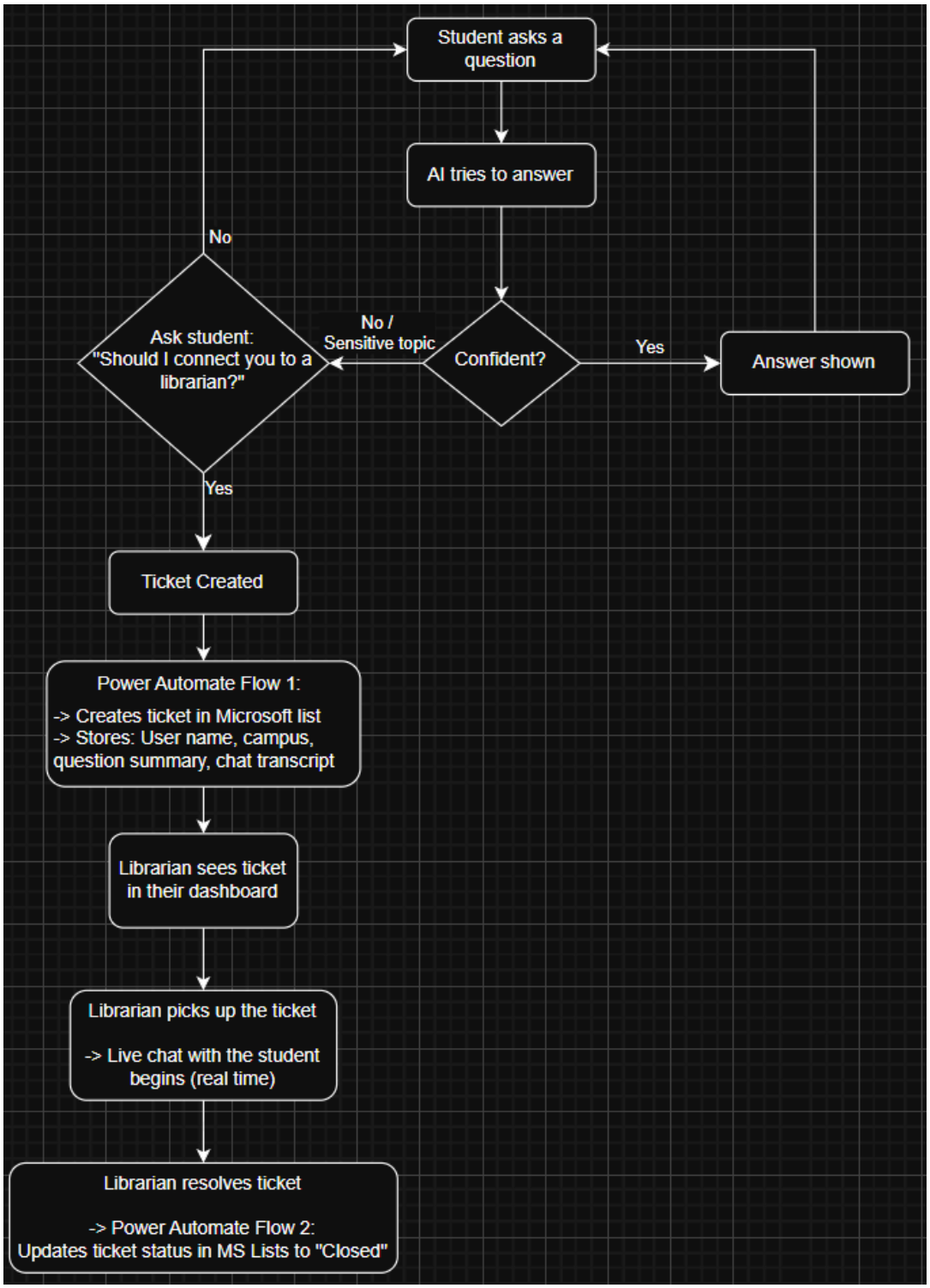
Components at a Glance

Component	Technology	Role
Frontend	React 19, TypeScript	Web interface for students and librarians
Backend	Python / FastAPI	Central coordinator: chat logic, auth, tickets
AI Service	Python / FastAPI + Groq	Understands questions, generates answers, detects escalation
Database	SQLite (prototype)	Stores users, chats, messages, tickets
Microsoft Power Automate	Power Automate Flows	Bridge to Microsoft Lists and Teams
Microsoft Lists	MS Lists	Ticket tracking for librarians

In production, **SQLite** would be replaced with **PostgreSQL**. The three services run independently and communicate via HTTP/WebSocket, making them independently deployable and scalable.

How Chatbot, Ticketing and Microsoft Are Connected

User flow (Authenticated)



What Goes Into a Ticket

When a ticket is created in Microsoft Lists, it contains:

- Student name, student number, campus
- Question category (e.g., "account", "fines", "research")
- Language (Dutch or English)
- AI-generated summary of the conversation
- Full conversation transcript
- Link to the live chat in the librarian dashboard

Guest Users

Students who are not logged in can also use the chatbot as a guest. If a guest needs to escalate, they can provide their email address. The system sends them a unique resume link so they can return to the same conversation later.

Microsoft Teams

Teams integration is handled via Power Automate. When a ticket is created in Microsoft Lists, a Power Automate automation can forward a notification to a Teams channel. This is configured on the Microsoft side and does not require changes to the prototype code.

The AI Model

Which model?

The prototype uses **Meta's Llama 3.1 (8 billion parameter version)**, accessed via the **Groq** cloud API.

Property	Value
Model	Llama 3.1 8B Instant
Provider	Groq (cloud API)
Type	Large Language Model (LLM)
Languages	English and Dutch
Response Time	< 1 second (Groq's hardware is optimized for speed)

Why this model?

Reason	Explanation
Speed	Groq specializes in fast inference. Responses in under a second, which feels natural in a chat interface
Open-source	Llama is released by Meta under an open license. No vendor lock-in, can be self-hosted
Cost	Free tier is sufficient for a prototype. Costs are very low at scale compared to GPT-4
Suitable size	8B parameters is large enough for structured Q&A with a good knowledge base, small enough to run cheaply
No training required	The model works well out-of-the-box when given good instructions and context (see 'How the AI Was Configured')

Advantages

- Fast, conversational responses
- Handles both Dutch and English (detected automatically)
- Does not hallucinate if properly grounded with a knowledge base
- Can be self-hosted on a private server for full data sovereignty
- Low operational cost

Limitations

- **Not as powerful as GPT-4/Claude for complex reasoning:**
  - But for structured library Q&A, it is sufficient
- **Session memory**:
  - The model has no persistent memory between conversations; each session starts fresh
- **Hallucination risk:**
  - Without a knowledge base, LLMs can confidently give wrong answers. We mitigate this by always providing relevant library documents as context
- **Dutch quality**:
  - Llama 3.1 performs better in English than Dutch, though performance is acceptable for the library use case
- **Not fine-tuned:**
  - The model has not been retrained on library-specific data (this was intentional: see Section 'How the AI Was Configured')
- **Cloud dependency**:
  - Currently uses Groq's cloud API. For full data privacy, a self-hosted setup would be needed

Choosing an AI Model

Key Considerations

If KdG's IT department wants to develop this further, here are the most important factors when selecting an AI model.

Privacy & Data Sovereignty

The most critical consideration for an institution. Every message a student sends is processed by the AI model. If using a cloud API (OpenAI, Anthropic, Groq), that data passes through an external server.

Options:

- **Cloud API (current):**
  - Fast, cheap, easy to set up
  - Data leaves the institution's network
- **Self-hosted (e.g., Ollama + Llama on-premise):**
  - All data stays on campus
  - Requires a server with a GPU
  - Higher setup cost, full control
- **Azure OpenAI Service:**
  - Microsoft's hosted version of GPT-4 with EU data residency options
  - May align better with existing KdG Microsoft contracts and GDPR requirements.

Language Support

The chatbot needs solid Dutch (NL) support. Models ranked by Dutch quality:

1. GPT-4 / GPT-4o (best for Dutch, but expensive and cloud-only)
2. Llama 3.1 70B (very good)
3. Llama 3.1 8B (**current**, good for structured Q&A)
4. Mistral 7B (originally considered, good Dutch)

Cost Model

- **Free/cheap API:**
  - Groq, Mistral API: suitable for prototype and low-volume production
- **Mid-range:**
  - Azure OpenAI GPT-3.5: good balance of cost and quality
- **Premium:**
  - GPT-4o, Claude 3 Opus: highest quality, highest cost
- **Self-hosted**
  - High upfront cost (GPU server), zero per-request cost after setup

Accuracy vs Speed Trade-off

Larger models are smarter but slower and more expensive. For a helpdesk chatbot:

- Library FAQ questions are relatively simple and well-defined
- An 8–13 billion parameter model is likely sufficient
- A 70B parameter model would handle edge cases better but cost significantly more

Integration Complexity

All modern LLMs offer an API compatible with OpenAI's standard. Switching providers (e.g., from Groq to Azure) is a configuration change, not a rewrite.

How the AI Was Configured

The AI model was not trained or fine-tuned. Instead, it was configured through three techniques: a system prompt, a knowledge base, and escalation rules.

System Prompt | The AI's Instructions

The AI receives a set of instructions at the start of every conversation.
It tells the model:

- **What it is:**
  - "You are the KdG Digital Library Helpdesk assistant"
- **What it can help with**:
  - opening hours, borrowing rules, accounts, finding sources, referencing
- **What it cannot do:**
  - modify accounts, confirm real-time availability, access library systems
- **Rules it must follow:**
  - Only use information from the provided knowledge base, do not invent policies
  - If a question is out of scope, say so clearly
  - If key information is missing (e.g., which campus), ask one short follow-up question
  - Always escalate for:
    - blocked accounts, fine disputes, unregistered returns, research consultations, database help
  - Keep answers concise (3-8 sentences)
  - Always reply in the student's language (Dutch or English - never mix)

Knowledge Base | The AI's Reference Library

The AI does not rely on its general training. Before answering, it searches a **knowledge base** of library-specific documents and uses them as context. This is called **RAG (Retrieval-Augmented Generation).**

The knowledge base consists of 7 markdown files covering:

Document	Content
opening_hours.md	**Opening hours per campus, exam period variations, summer hours**
borrow_rules.md	**Loan periods, renewal rules, limits per campus, fine calculations**
account_access.md	**How to handle blocked accounts, login issues**
database_remote_access.md	**VPN setup, remote access to library databases**
referencing_citation.md	**APA referencing basics, citation guidance**
functional_input.md	**System scope, boundaries, escalation guidelines**
intents.json	**12 predefined question patterns for intent matching**

How RAG works: When a student asks a question, the system searches the knowledge base for the most relevant paragraphs and gives them to the AI as context. The AI then formulates an answer based only on that material, not on guesswork.

When a user sends a query:

1. The query is embedded
2. ChromaDB is queried for similar vectors
3. Relevant knowledge chunks are retrieved
4. These chunks are injected into the prompt
5. The LLM generates a response

This ensures responses are grounded in the knowledge base rather than generated from general model knowledge.

Intent Detection | What Is the Student Asking?

The system recognizes 12 types of questions (intents):

Intent	Handled by AI?	Escalate to librarian?
Opening Hours	**Yes**	**No**
Loan Periods	**Yes**	**No**
Loan Limits (per campus)	**Yes**	**No**
Remote Database access	**Yes**	**No**

Inter-library loans	**Yes**	**No**
Blocked account	**No**	**Always**
Fine disputes	**No**	**Always**
Return not registered	**No**	**Always**
Research help	**No**	**Always**
Basic APA referencing	**Yes**	**No**
Detailed APA review	**No**	**Always**
Database help	**No**	**Always**

Each intent contains:

- **intent:** identifier (e.g., opening_hours)
- **category:** type of question (practical, policy)
- **campus_required:** whether context is needed
- **escalation_required:** whether to escalate
- **required_fields:** additional required inputs
- **description:** explanation of the intent
- **examples:** sample user queries

This allows the system to classify user queries before retrieval.

Intent Example

Example intent structure:

- Intent: opening_hours
- Category: practical
- Campus required: false
- Escalation required: false
- Description: Questions about general library opening

Examples include:

- What are the library opening hours?
- Is the library open today?
- Wat zijn de openingsuren van de bibliotheek?

This enables multilingual detection and consistent classification.

Escalation Rules | When to Involve a Librarian?

Escalation is determined by a combination of rules (not just the AI's judgment):

1. **Always-escalate intents**
    a. Certain question types always require a librarian (see table above)
2. **Low confidence**
    a. If the AI's confidence score is below a threshold, it escalates rather than guessing
3. **Sensitive categories**
    a. Account and fine-related questions trigger extra caution
4. **Repeated failed attempts**
    a. If the AI cannot help after two tries, it escalates
5. **Out of scope**
    a. If the question is unrelated to the library, it says so and offers escalation

Decision logic is implemented using DMN rules inside the AI service.

DMN determines:

- Whether the AI should answer
- Whether escalation is required
- Whether additional information is needed

This separates decision logic from knowledge retrieval.

Language Detection

Language is detected automatically using keyword matching. Dutch keywords (e.g., "lenen", "boete", "openingsuren") trigger Dutch mode. The AI then responds entirely in the detected language.

Storage Architecture

Knowledge base content is stored in two layers:

- **Backend Storage:**
    - Stores KB files (title, slug, structured sections)
- **Vector Database (ChromaDB):**
    - Stores embeddings of each section
    - Enables semantic similarity search

This ensures persistent storage and efficient retrieval.

Ingestion Process

When a knowledge base file is created or updated:

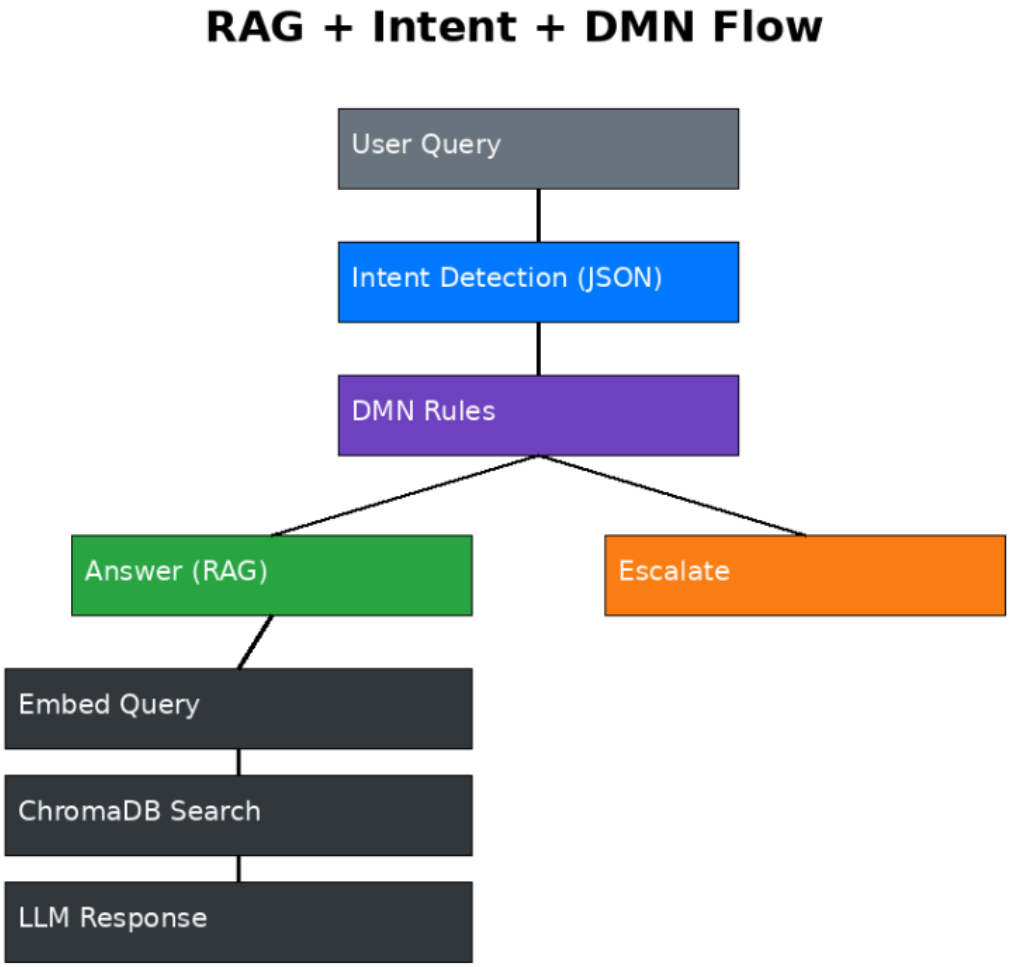
- Content is sent to the backend
- It is split into structured sections
- Each section is converted into embeddings
- Embeddings are stored in ChromaDB

A manual POST /ingest endpoint is used during startup to load all knowledge into the vector database.

System Flow (Combined)

The full processing flow combines intent detection, DMN rules, and RAG retrieval:

- User query received
- Intent classification (JSON intents)
- DMN rules evaluation
- If allowed → RAG retrieval
- If required → escalation



This hybrid approach ensures both accuracy and control.

Design Summary

- Maintainability: knowledge editable via admin UI
- Scalability: new intents and KB entries can be added easily
- Separation of concerns: backend, AI, DMN
- Accuracy: grounded responses via RAG
- Control: deterministic decisions via DMN

Setup & Configuration

What Is Needed to Run This

Requirement	Details
Python 3.11+	**For backend and AI service**
Node.js 20+	**For frontend**
Groq API key	**Free at console.groq.com (for AI responses)**
Microsoft Power Automate	**3 flows configured (create ticket, close ticket, send email)**
Microsoft Lists	**One list configured to receive tickets**

Environment Configuration

Each service has a .env configuration file. Key settings:

- **Backend (backend/.env):**
  - DATABASE_URL=sqlite:///./helpdesk.db
    - **Database location**
  - AI_SERVICE_URL=http://localhost:8001
    - **Where AI service runs**
  - POWER_AUTOMATE_FLOW1_URL=<your-flow-url>
    - **Create ticket flow**
  - POWER_AUTOMATE_FLOW2_URL=<your-flow-url>
    - **Close ticket flow**
  - POWER_AUTOMATE_FLOW3_URL=<your-flow-url>
    - **Send guest email flow**
  - JWT_SECRET=<a-long-random-string>
    - **Security key for auth**
  - FRONTEND_URL=http://localhost:5173
    - **For guest email links**

- **AI Service (ai/.env):**

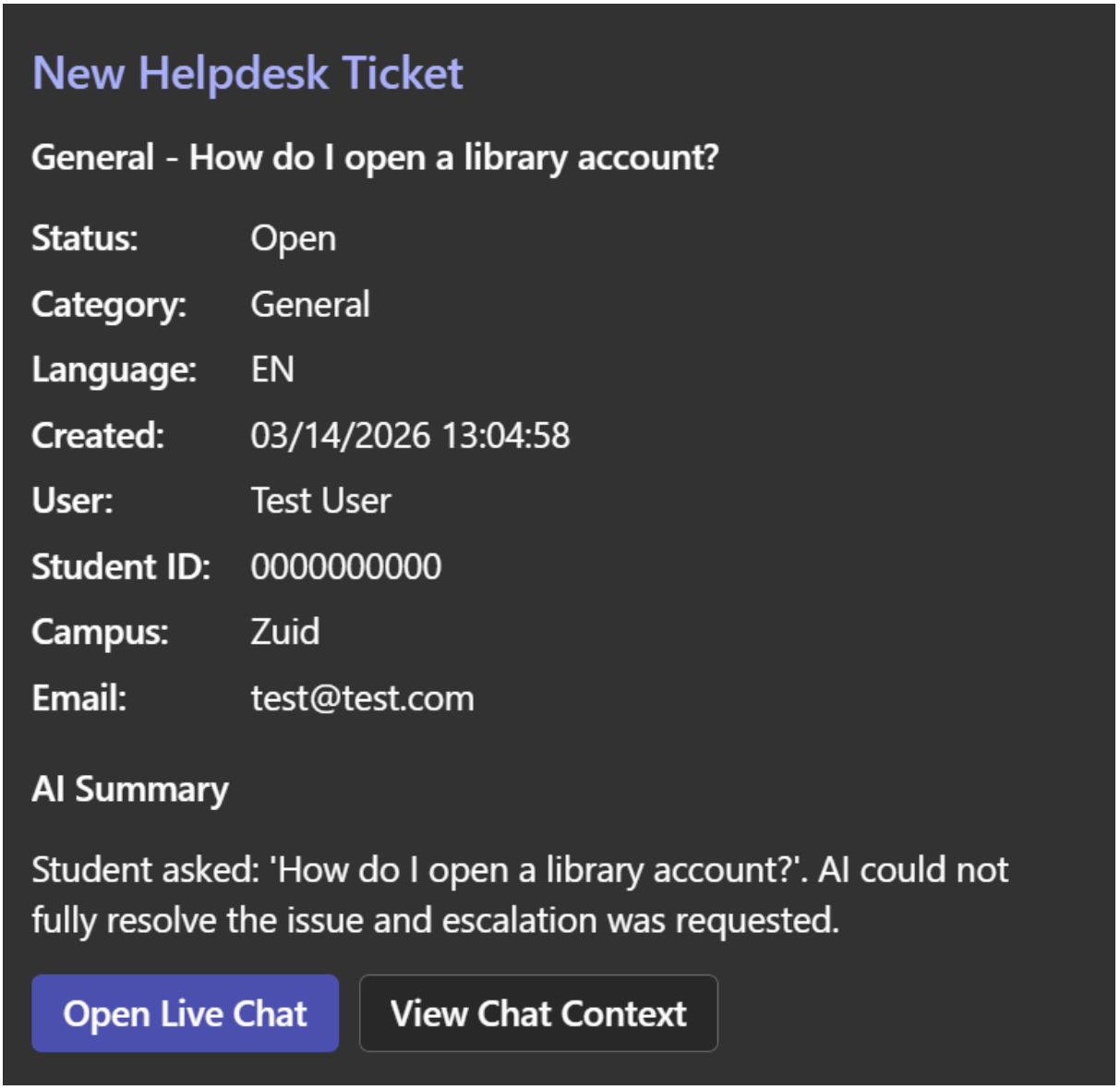
- GROQ_API_KEY=<your-groq-api-key>
  - **Groq cloud API key**
- GROQ_MODEL=llama-3.1-8b-instant
  - **Which model to use**

- **Frontend (frontend/.env):**
  - VITE_API_URL=http://localhost:8000
    - **Where backend runs**

Microsoft Teams

A Teams channel is required to receive ticket notifications. When a student escalates, Flow 1 posts an Adaptive Card to this channel containing:

- **Ticket title, Status, Category, Language, Created At (Data + Time), Username, Student ID, Campus, Email, AI Summary**
- **Two action buttons:**
  - "Open Live Chat" (links to the librarian dashboard)
  - "View Full Context" (links to the SharePoint document)



Setup options

1. Create a dedicated channel (e.g., "Library Helpdesk Tickets") in the librarians' existing Teams team
2. Use an existing channel. The Power Automate flow posts to whichever channel is configured

Configuration

The target channel is set inside the Power Automate flow itself (not in the prototype code). When setting up the flows, make sure to use the correct channel(s) in the configuration.

Microsoft Lists

One list is required to track all helpdesk tickets. It must be created inside a SharePoint site or a Teams team that the librarians have access to.

Required Lists columns

Column	Type	Notes
Ticket ID	**Single line of text**	- Enforce unique values
Title	**Single line of text**	
AI Summary	**Multiple lines of text**	
Category	**Choice**	- Lending, Account, Research, Academic Writing, Open Hours, General - Drop-Down menu - Allow multiple selections
Status	**Choice**	- Open, In Progress, Closed - Drop-Down menu
Language	**Choice**	- NL, EN - Drop-Down menu
Dashboard Link	**Hyperlink**	
Chat Context	**Hyperlink**	
User Name	**Single line of text**	
Student ID	**Single line of text**	
Campus	**Single line of text**	
Created Date	**Date and time**	- Include Time

		- Friendly format
User Email	**Single line of text**	
Teams Message ID	**Single line of text**	- Hidden (Required for updating the adaptive card)

Three views to configure

1. **All Tickets**
    a. Default view, all columns visible, sorted by Created (newest first)
2. **Open & Active**
    a. Filtered to Status = "Open" or "In Progress"
    b. For librarians checking what needs attention
3. **Closed:**
    a. Filtered to Status = "Closed"
    b. For historical reference and reporting

Microsoft SharePoint

A SharePoint document library is required to store the full chat context (before escalation) for each escalated ticket. Flow 1 automatically uploads a formatted text document per ticket.

Setup

- Create a document library named "Chat Contexts" (or equivalent) in the same SharePoint site as the Lists list
- Grant librarians read access to the library
- Flow 1 uploads the context document and stores its link in the corresponding Lists ticket item

Document content

Each file contains the full formatted conversation transcript including the AI summary, student details, and all messages exchanged before escalation. This gives librarians the full picture before joining the live chat.

Microsoft Power Automate Flows

Three Power Automate HTTP-triggered flows are required. Each is triggered by the backend via an HTTP POST. The webhook URLs must be configured as environment variables.

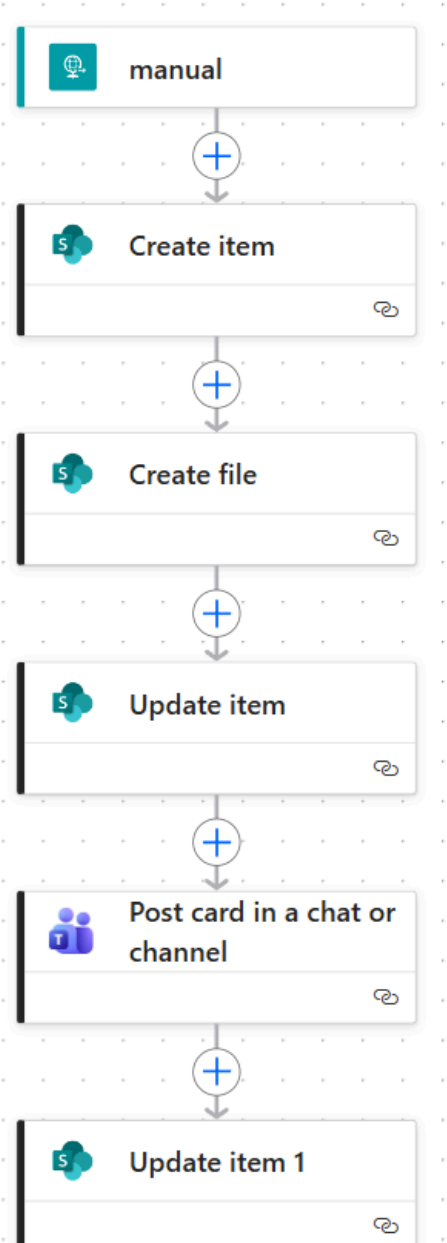
Flow	Trigger	Action
Flow 1	**Backend calls when student escalates**	**Creates a new item in Microsoft Lists, returns the item ID**

Flow 2	**Backend calls when librarian resolves**	**Updates item status in Microsoft Lists to "Closed"**
Flow 3	**Backend calls when guest escalates with email**	**Sends resume-link email via Outlook**

These flows need to be created and configured in the KdG Power Automate environment. The prototype code only needs the webhook URLs.

Flow 1

1. Select **"Automated Cloud Flow"** and **Skip** the popup.
2. For **Trigger** select **When an HTTP request is received** (Request)
    a. Who can trigger the flow?
        i. Based on the enterprise settings
        ii. We used **Anyone**
    b. Request Body JSON Schema
        i. See Section Flow 1 JSON
3. **Create item** (SharePoint)
    a. Site Address
        i. The Teams channel
    b. List Name
        i. MS Lists list name
    c. Advanced parameters
        i. See Section Flow 1 Parameters 1.1
4. **Create file** (SharePoint)
    a. Site Address
        i. The Teams channel
    b. Folder Path
        i. Chat context folder path
    c. File Name
        i. @{concat('ticket-', triggerBody()?['chat_id'], '-context.txt')}
    d. File Content
        i. @{triggerBody()?['context_file_content']}
5. **Update item** (SharePoint)
    a. Site Address
        i. The Teams channel
    b. List Name
        i. MS Lists list name
    c. Id
        i. @{outputs('Create_item')}



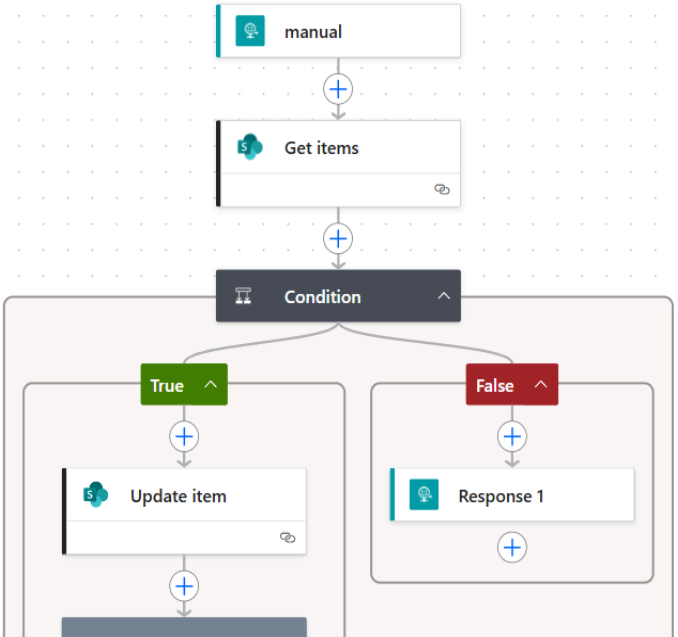
?['body/ID']}

d. Advanced parameters
   i. See Section Flow 1 Parameters 1.2

6. **Post card in a chat or channel** (Teams)
   a. Post as
      i. Flow bot
   b. Post in
      i. Channel
   c. Team
      i. The Teams team
   d. Channel
      i. The Teams team channel
   e. Adaptive Card
      i. See Section Flow 1 Parameters 1.3
7. **Update item** (SharePoint)
   a. Site Address
      i. The Teams channel
   b. List Name
      i. MS Lists list name
   c. Id
      i. @{outputs('Create_item')?['body/ID']}
   d. Advanced parameters
      i. Teams Message ID
         1. @{outputs('Post_card_in_a_chat_or_channel')?['body/id']}
8. **Response** (Request)
   a. Status Code
      i. 200
   b. Headers
      i. Content-Type | application/json
   c. Body
      i. {
         "ticket_id": "@{outputs('Create_item')?['body/ID']}",
         "status": "created"
         }

Flow 2

1. Select **"Automated Cloud Flow"** and **Skip** the popup.
2. For **Trigger** select **When an HTTP request is received** (Request)
   a. Who can trigger the flow?
      i. Based on the enterprise settings



        ii. We used **Anyone**
    b. Request Body JSON Schema
        i. See Section [Flow 2 JSON]

3. **Get items** (SharePoint)
    a. Site Address
        i. The Teams team
    b. List Name
        i. MS Lists list name
    c. Advanced parameters
        i. Filet Query
            1. TicketID eq '@{triggerBody()?['ticket_id']}'
        ii. Top Count
            1. 1
4. **Condition** (Control)
    a. Condition expression
        i. And
        ii. @{length(body('Get_items')?['value'])} | is greater than | 0

If True

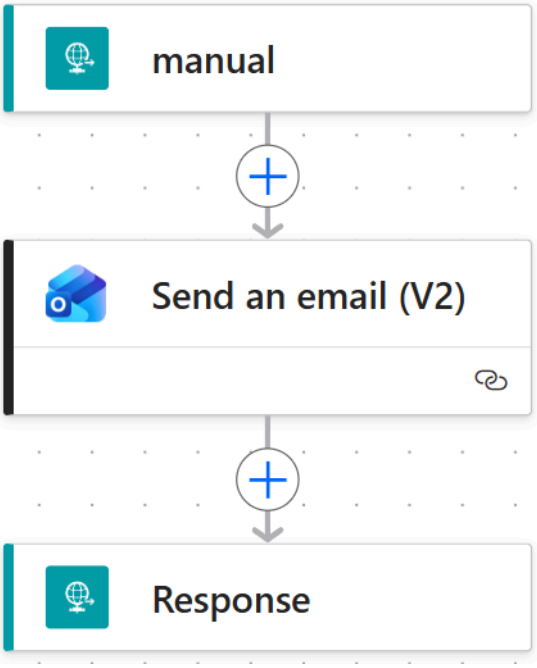
5. **Update item** (SharePoint)
    a. Site Address
        i. The Teams channel
    b. List Name
        i. MS Lists list name
    c. Id
        i. @{first(body('Get_items')?['value'])?['ID']}
    d. Advanced parameters
        i. Status Value
            1. Closed
6. **Update an adaptive card in chat or channel** (Teams)
    a. Post as
        i. Flow bot
    b. Post in
        i. Channel
    c. Message ID
        i. @{items('For_each')?['TeamsMessageID']}
    d. Team
        i. The Teams Team channel
    e. Channel
        i. The Teams team channel
    f. Adaptive Card

        i. See Section Flow 2 Parameters
7. **Response** (Request)
    a. Status Code
        i. 200
    b. Headers
        i. Content-Type | application/json
    c. Body
        i. {
           "status": "updated"
           }

If False

8. **Response** (Request)
    a. Status Code
        i. 404
    b. Headers
        i. Content-Type | application/json
    c. Body
        i. {
           "status": "error",
           "message": "Ticket not found"
           }

Flow 3



1. Select **"Automated Cloud Flow"** and **Skip** the popup.
2. For **Trigger** select **When an HTTP request is received** (Request)
    a. Who can trigger the flow?
        i. Based on the enterprise settings
        ii. We used **Anyone**
    b. Request Body JSON Schema
        i. See Section Flow 3 JSON
3. **Send an email V2** (Outlook)
    a. To
        i. @{triggerBody()?['user_email']}
    b. Subject
        i. @{triggerBody()?['email_subject']}
    c. Body
        i. @{triggerBody()?['email_body']}
4. **Response** (Request)
    a. Status Code
        i. 200

b. Body
   i. {
      "status": "sent"
      }

Starting the Services (Development)

Start the AI service (port 8001)

1. **cd ai**
2. **pip install -r requirements.txt**
3. **uvicorn src.app:app --port 8001**
4. **Then call: POST /ingest to load the knowledge base (once)**

Start the backend (port 8000)

1. **cd backend**
2. **pip install -r requirements.txt**
3. **uvicorn app.api.main:app --port 8000**

Start the frontend (port 5173)

1. **cd frontend**
2. **npm install**
3. **npm run dev**

Pre-configured Accounts

The backend seeds these accounts on first startup:

Username	Password	Role
admin	**admin123**	**Admin**

Registering

- Users register themselves via the web interface.
- To create a **librarian** account during development or testing, register any username prefixed with **'lib_'** (e.g., `lib_sarah`).
  - The system will automatically assign the librarian role.

IMPORTANT – Development / Test Only:

- The **'lib_'** prefix method is a dummy authentication mechanism for development and testing purposes ONLY.

- It must NOT be used in production, as any user could self-register as a librarian.
- In production, librarian accounts must be created through a controlled process.
    - Ideally integrated with KdG's existing identity system (e.g., Azure AD / Microsoft 365 accounts).

Current Status

Known Limitations of the Prototype

- **Database:**
    - SQLite is used for simplicity.
    - Production deployments should use PostgreSQL.
- **AI session memory:**
    - Conversation history within a session is maintained, but is lost if the AI service restarts.
- **Self-hosted vs cloud AI:**
    - Currently uses Groq cloud API.
    - For full data privacy compliance, the model can be run on-premise using Ollama (free, open-source).
- **Dutch AI quality:**
    - The 8B model performs slightly better in English.
    - Upgrading to a 70B model or GPT-4 would improve Dutch response quality.
- **Authentication:**
    - No password reset functionality.
    - JWT tokens do not expire (should be limited in production).

Technology and Technique Glossary

Term	Explanation
API	A way for software systems to talk to each other over the internet
Chunks/Chunking	Chunking is the process of splitting large pieces of text into smaller, manageable sections (chunks) so they can be processed, stored, and retrieved more efficiently.
ChromaDB	A database designed to store and search text by meaning, not just keywords. Used to find the most relevant knowledge base entries
DMN (Decision Model and Notation)	A standard format for expressing business rules as decision tables. Used for escalation logic
FastAPI	A Python framework for building fast web APIs. Similar to how WordPress builds websites, but for software-to-software communication
Hexagonal Architecture	A software design pattern where business logic is kept separate from external dependencies (database, APIs). Makes the system easier to test and modify
Ingestion	Ingestion is the process of taking raw data, processing it, and storing it in a system so it can be searched and used later.
JWT (JSON Web Token)	A digital ID card that proves a user is logged in. Stored in the browser
LLM (Large Language Model)	An AI model trained on large amounts of text, capable of understanding and generating human language
Power Automate	Microsoft's tool for automating workflows between Microsoft 365 services (Lists, Teams, Outlook)
React	A JavaScript library for building interactive web interfaces
RAG (Retrieval-Augmented Generation)	Giving the AI relevant documents to read before answering, so it uses real facts instead of guessing
SQLite / PostgreSQL	Database systems for storing structured data. SQLite is lightweight (single file), PostgreSQL is enterprise-grade
WebSocket	A persistent connection between browser and server that allows real-time messaging (like a phone call, not a letter)

Extras

Flow 1 JSON


```
{
  "type": "object",
  "properties": {
    "title": {
      "type": "string"
    },
    "summary": {
      "type": "string"
    },
    "category": {
      "type": "string"
    },
    "language": {
      "type": "string"
    },
    "last_messages": {
      "type": "string"
    },
    "chat_id": {
      "type": "string"
    },
    "dashboard_url": {
      "type": "string"
    },
    "user_name": {
      "type": "string"
    },
    "student_id": {
      "type": "string"
    },
    "campus": {
      "type": "string"
    },
    "user_email": {
      "type": "string"
    }
  }
}
```

Flow 1 Parameters 1.1

- **Ticket ID**
  - @{triggerBody()?['chat_id']}
- **Title**
  - @{triggerBody()?['title']}
- **AI Summary**
  - @{triggerBody()?['summary']}
- **Status Value**
  - Open
- **Language Value**
  - @{triggerBody()?['language']}
- **Dashboard LInk**
  - @{triggerBody()?['dashboard_url']}
- **User Name**
  - @{triggerBody()?['user_name']}
- **Student ID**
  - @{triggerBody()?['student_id']}
- **Campus**
  - @{triggerBody()?['campus']}
- **Created Date**
  - @{utcNow()}
- **User Email**
  - @{triggerBody()?['user_email']}
- **Category**
  - @{triggerBody()?['category']}

Flow 1 Parameters 1.2

- **Ticket ID**
  - @{outputs('Create_item')?['body/ID']}
- **Title**
  - @{outputs('Create_item')?['body/Title0']}
- **Chat Context Link**
  - @{concat('https://studentkdg.sharepoint.com/sites/TestLAB/Chat Contexts/ticket-', triggerBody()?['chat_id'], '-context.txt')}

Flow 1 Parameters 1.3

```
{
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "type": "AdaptiveCard",
```

```
"version": "1.4",
"speak": "New helpdesk ticket received",
"body": [
  {
    "type": "TextBlock",
    "text": "New Helpdesk Ticket",
    "weight": "Bolder",
    "size": "Large",
    "color": "Accent"
  },
  {
    "type": "TextBlock",
    "text": "@{triggerBody()?['title']}",
    "weight": "Bolder",
    "size": "Medium",
    "wrap": true
  },
  {
    "type": "FactSet",
    "facts": [
      { "title": "Status:",     "value": "Open" },
      { "title": "Category:", "value": "@{triggerBody()?['category']}" },
      { "title": "Language:", "value": "@{triggerBody()?['language']}" },
      { "title": "Created:", "value": "@{utcNow()}" },
      { "title": "User:", "value": "@{triggerBody()?['user_name']}" },
      { "title": "Student ID:", "value": "@{triggerBody()?['student_id']}" },
      { "title": "Campus:", "value": "@{triggerBody()?['campus']}" },
      { "title": "Email:", "value": "@{if(empty(triggerBody()?['user_email']), '-',
triggerBody()?['user_email'])}" }
    ]
  },
  {
    "type": "TextBlock",
    "text": "AI Summary",
    "weight": "Bolder",
    "spacing": "Medium"
  },
  {
    "type": "TextBlock",
    "text": "@{triggerBody()?['summary']}",
    "wrap": true
  }
],
"actions": [
```

```
      {
         "type": "Action.OpenUrl",
         "title": "Open Live Chat",
         "url": "@{triggerBody()?['dashboard_url']}",
         "style": "positive"
      },
      {
         "type": "Action.OpenUrl",
         "title": "View Chat Context",
         "url": "https://studentkdg.sharepoint.com/sites/TestLAB/Chat%20Contexts/ticket-@{triggerBody()?['chat_id']}-context.txt"
      }
   ]
```

Flow 2 JSON

```
{
   "type": "object",
   "properties": {
      "ticket_id": {
         "type": "string"
      },
      "status": {
         "type": "string"
      }
   },
   "required": [
      "ticket_id",
      "status"
   ]
}
```

Flow 2 Parameters

```
{
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "type": "AdaptiveCard",
    "version": "1.4",
    "body": [
       {
          "type": "TextBlock",
```

```
            "text": "Helpdesk Ticket — Resolved",
            "weight": "Bolder",
            "size": "Large",
            "color": "Good"
        },
        {
            "type": "TextBlock",
            "text": "@{items('For_each')?['Title0']}",
            "weight": "Bolder",
            "size": "Medium",
            "wrap": true
        },
        {
            "type": "FactSet",
            "facts": [
                { "title": "Status:",     "value": "Closed" },
                { "title": "Category:",   "value": "@{items('For_each')?['Category']?['Value']}" },
                { "title": "Language:",   "value": "@{items('For_each')?['Language']?['Value']}" },
                { "title": "Created:",    "value": "@{items('For_each')?['Created']}" },
                { "title": "User:",       "value": "@{items('For_each')?['UserName']}" },
                { "title": "Student ID:", "value": "@{items('For_each')?['StudentID']}" },
                { "title": "Campus:",     "value": "@{items('For_each')?['Campus']}" },
                  { "title": "Email:",      "value": "@{if(empty(items('For_each')?['UserEmail']), '-',
items('For_each')?['UserEmail'])}" }
            ]
        },
        {
            "type": "TextBlock",
            "text": "AI Summary",
            "weight": "Bolder",
            "spacing": "Medium"
        },
        {
            "type": "TextBlock",
            "text": "@{items('For_each')?['AISummary']}",
            "wrap": true
        }
    ],
    "actions": [
        {
            "type": "Action.OpenUrl",
            "title": "View Resolved Chat",
            "url": "@{items('For_each')?['DashboardLink']}",
            "style": "positive"
```

```
        },
        {
            "type": "Action.OpenUrl",
            "title": "View Chat Context",
            "url": "@{items('For_each')?['ChatContextLink']}"
        }
    ]
}
```

Flow 3 JSON

```
{
  "type": "object",
  "properties": {
    "user_email": {
      "type": "string"
    },
    "user_name": {
      "type": "string"
    },
    "resume_url": {
      "type": "string"
    },
    "email_subject": {
      "type": "string"
    },
    "email_body": {
      "type": "string"
    }
  }
}
```